

CHAP 13(1): pandas 라이브러리

pandas 개요, Series 자료형, Series 객체 생성,
DataFrame 자료형, DataFrame 객체 생성,
DataFrame 데이터 추출, DataFrame 연산 함수,
외부 데이터 파일 읽기, CSV파일 읽기,
엑셀 파일 읽기,
DataReader로 Web 데이터 가져오기

[참고도서]

- 데이터 분석을 위한 파이썬 철저 입문
- 파이썬 Express
- 따라하며 배우는 파이썬과 데이터과학
- 으뜸 데이터분석과 머신러닝

pandas 개요

- pandas는 데이터 분석(Data Analysis)을 위한 파이썬 라이브러리. 테이블 형태의 데이터에 대한 분석·연산에 적합
- pandas의 자료구조
 - Series. 1차원 자료구조(1열 자료, 인덱싱된 1차원 배열)
 - DataFrame. 2차원 자료구조
 - 행(index)과 열(column)로 구성
 - 행과 열에 레이블이 있어 각 행과 열에 접근 가능.
 - 파이썬 리스트, 딕셔너리, 넘파이 배열을 쉽게 변환 가능
 - CSV 파일, 엑셀 파일, web 데이터 등을 읽고 변환 가능.
- pandas에서 Series, DataFrame 등의 객체를 만든 후, 다양한 방법을 통해 데이터를 액세스할 수 있다.
 - pandas 자료구조는 **행과 열에 레이블이 있으며,** **인덱싱** 혹은 **속성(Attribute)**을 사용하여 데이터를 액세스.
 - **부울린 인덱싱 (boolean indexing)**을 사용하여 특정 조건의 데이터만 필터링 할 수 있다.
 - Matplotlib를 이용하여 데이터를 시각화할 수 있다.

pandas 개요

- pandas는 데이터에 대한 다양한 연산 함수들을 제공.
 - 평균, 개수, 데이터프레임의 열 사이의 상관 관계 등을 계산
- pandas는 다양한 외부 리소스의 데이터를 읽고 처리하는 기능을 제공.
 - CSV 파일, 텍스트 파일, 엑셀 파일, **web** 데이터 등을 읽고
 - Matplotlib를 이용하여 데이터를 시각화할 수 있다.
- pandas를 설치하고, pandas를 import해야 사용할 수 있다.
 - 윈도우 콘솔창(커맨드창)에서 **install**을 실행

```
pip install pandas # install 명령어
```
 - pandas 라이브러리를 import하고 사용한다

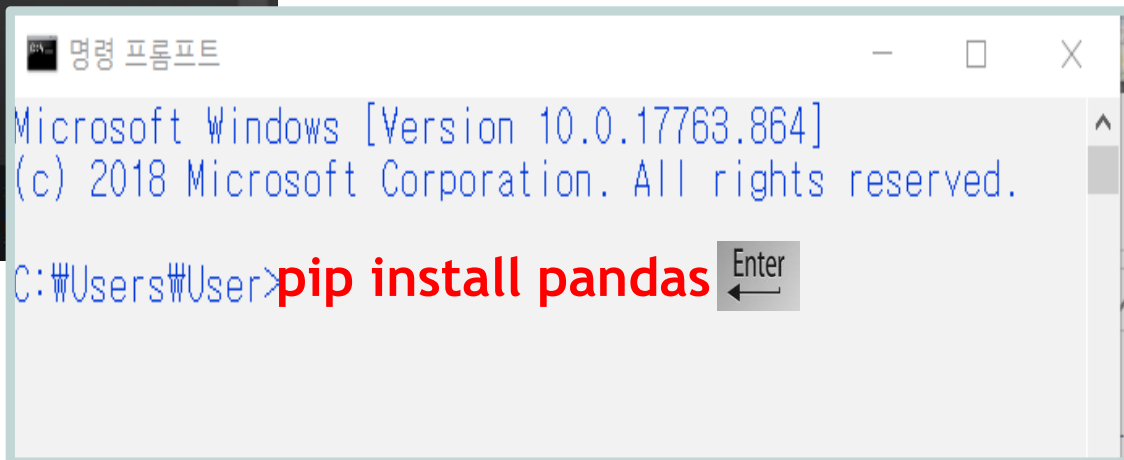
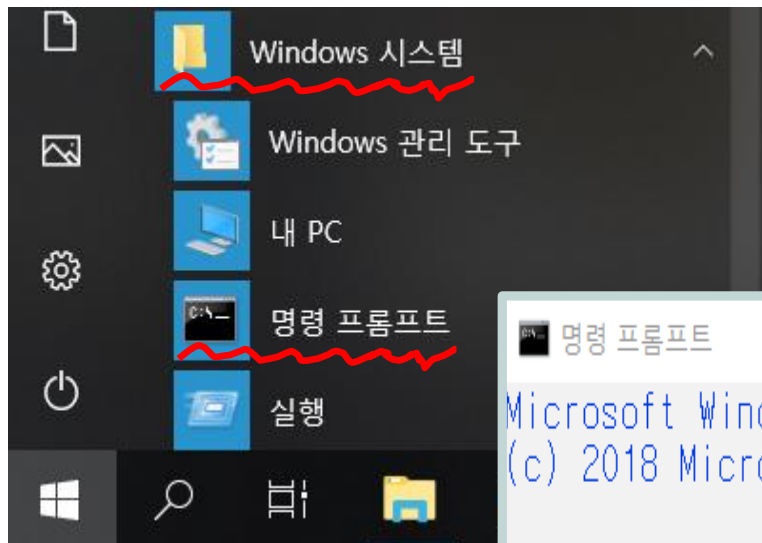
```
import pandas as pd # import 문. 별칭
```

pd를 지정

pandas 개요

- pandas를 윈도우 커맨드창에서 install하기
 - 윈도우 콘솔창은 <Windows 시스템> → <명령 프롬프트>

pip install pandas **# install 명령어**



Series 자료형

- Series는 간단한 1차원 자료구조.
 - index가 있는 1차원 배열로 볼 수 있음.
 - 리스트와 같은 일련의 시퀀스 데이터를 받아들이며, 인덱스가 있는 객체를 생성.
- 별도의 인덱스 레이블을 지정하지 않으면, 0부터 시작되는 디폴트 정수 인덱스를 사용.
- Series 객체의 속성을 통해 인덱스와 값을 확인 할 수 있다.
 - index 인덱스를 갖고 있음
 - values 값을 갖고 있음
- 예1) Series 객체 생성
 - 디폴트 정수 인덱스를 사용하기.

```
import pandas as pd
```

```
data = [1, 3, 5, 7, 9]
```

```
s = pd.Series(data) # 인덱스는 디폴트 값 사용  
print(s)
```

출력 화면

index

0	1
1	3
2	5
3	7
4	9

dtype: int64

Series 객체 생성

- 예2) 인덱스 레이블을 지정하기

```
import pandas as pd
```

```
data = [1, 3, 5, 7, 9]
```

```
s = pd.Series(data, index= ['영희', '순희', '길동', '지성', 'Bob'] )
```

```
print(s)
```

인덱스 레이블 지정

index

출력 화면

영희	1
순희	3
길동	5
지성	7
Bob	9

dtype: int64

- 예2) 속성으로 데이터 액세스하기
 - values, index

```
print(s.values) # 값을 액세스
```

```
print(s.index) # 인덱스를 액세스
```

출력 화면

```
[1 3 5 7 9] # 값
```

```
Index(['영희', '순희', '길동', '지성', 'Bob'], dtype='object') # 인덱스
```

Series 객체 생성

- 예3) 사전 객체를 사용하여 **Series** 객체 생성.
 - 사전의 키를 인덱스로, 사전의 값을 값(**values**)으로 구성한다.

Series 객체를 생성

```
input2 = pd.Series( {"England": "London",  
                    "India": "New Delhi", "USA": "Washington",  
                    "Belgium": "Brussels"} )
```

```
print(input2)
```

index **England** London
India New Delhi
USA Washington
Belgium Brussels
dtype: object

출력 화면

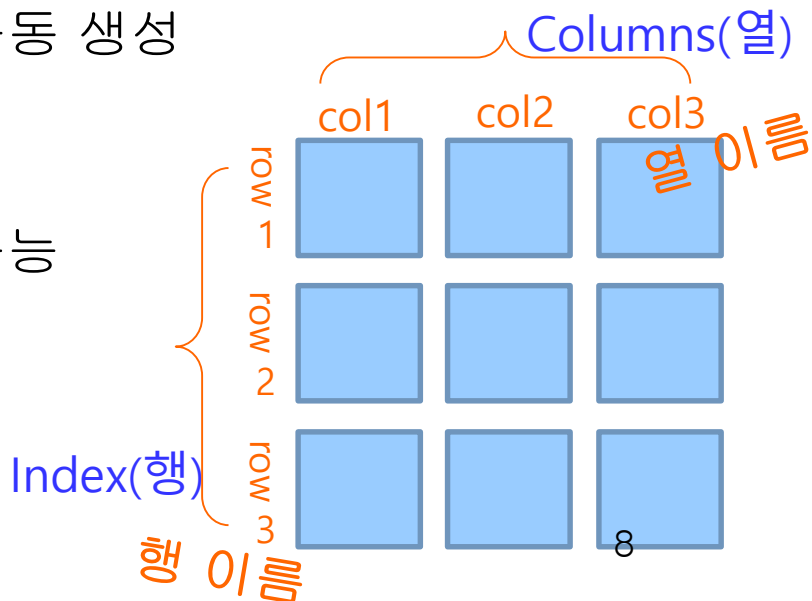
<i>index</i>	<i>values</i>
England	London
India	New Delhi
USA	Washington
Belgium	Brussels

생성된 Series 객체

DataFrame 자료형

- 2차원 자료구조.
- DataFrame는 행과 열이 있는 테이블 데이터(Tabular Data)를 받아들임.
 - index와 column에 각각 레이블(이름)이 있다.
 - column별로 다른 데이터 타입을 가질 수 있음.
 - 기본적으로 column단위로 데이터를 관리.
- DataFrame 객체는 다양한 데이터 타입을 사용해서 만들 수 있다.
 - ndarray, dictionary, series, list
 - `pd.DataFrame()`을 사용해서 생성.
 - index는 0부터 시작하는 정수로 자동 생성
 - index 키워드로 인덱스 이름을 지정 가능.
 - columns 키워드로 열 이름 지정 가능
- 각 행은 하나의 객체에 대한 정보를 표시
- 각 열은 서로 다른 속성을 나타냄

*2차원 3행 3열의
DataFrame



DataFrame 객체 생성

- 예1) Data Frame 객체 생성
 - index 이름, 열 이름으로 디폴트 값을 사용

```
import pandas as pd
```

```
g_data = [ [2016, 2.8, '1.63M'], [2017, 3.1, '1.73M'],  
           [2018, 3.0, '1.83M'] ]
```

```
df1 = pd.DataFrame( g_data )  
print(df1)
```

😊 출력 화면

행 이름 (디폴트 값)

열 이름 (디폴트 값)

	0	1	2
0	2016	2.8	1.637M
1	2017	3.1	1.73M
2	2018	3.0	1.83M

- index 이름, 열 이름 지정

```
df2 = pd.DataFrame( g_data, index= [1, 2, 3],  
                    columns= ['year', 'GDP_rate', 'GDP' ] )  
print(df2)
```

😊 출력 화면

행 이름

열 이름

	year	GDP_rate	GDP
1	2016	2.8	1.63M
2	2017	3.1	1.73M
3	2018	3.0	1.83M

DataFrame 객체 생성

- 예2) Dictionary 데이터 → Data Frame 자료구조로 변환
 - dict의 Key를 **열(column) 이름**으로,
 - dict의 Value를 **열의 값**으로 함

```
import pandas as pd
```

```
g_data = { 'year': [2016,2017,2018], 'GDP_rate': [2.8, 3.1, 3.0],  
           'GDP': ['1.63M','1.73M','1.83M']} # 사전 객체
```

```
df= pd.DataFrame(g_data) # df= pd.DataFrame( data= g_data )  
print(df)
```

출력 화면

	year	GDP_rate	GDP
0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	3.0	1.83M

columns(열 이름)

	col1	col2	col3
	year	GDP_rate	GDP
0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	0.0	1.83M

index

rows(행)

row1

row2

row3

DataFrame 객체 생성

- 예3) Dictionary 데이터 → Data Frame 자료구조로 변환
 - dict의 Key를 **열(column) 이름**으로, dict의 Value를 **열의 값**으로 함
 - Series의 index를 **index(행 이름)**로 함.

```
import pandas as pd
s1= pd.Series([2016,2017,2018]) #시리즈 객체
s2= pd.Series([2.8, 3.1, 3.0])   #시리즈 객체
s3= pd.Series(['1.63M','1.73M','1.83M']) #시리즈 객체
g_data = { 'year': s1, 'GDP_rate': s2, 'GDP': s3} # 사전 객체

df= pd.DataFrame(g_data) # df= pd.DataFrame( data= g_data )
print(df)
```

출력 화면

	year	GDP_rate	GDP
0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	3.0	1.83M

column(열 이름)

	year	GDP_rate	GDP
0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	0.0	1.83M

index

DataFrame 객체 생성

- 예제2 에서 생성된 DataFrame 객체 df에는

- 3개의 column(열)이 있다.

- ‘year’, ‘GDP_rate’, ‘GDP’ #열 이름
- 데이터 인덱싱에 열(column) 이름이 사용됨

columns(열 이름)

	year	GDP_rate	GDP
index 0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	0.0	1.83M

시리즈

- 3개의 Series 객체가 있다.

- 열(column) ‘year’, ‘GDP_rate’, ‘GDP’의 값은 각각 Series 객체.

- column을 조회할 때는 df['열 이름'], df.속성

- 속성은 열 이름과 동일.
- 3개의 열 이름 ‘year’, ‘GDP_rate’, ‘GDP’ 을 인덱싱에 사용하여 해당 Series 객체에 접근할 수 있다.
- 예)

```
df.GDP      # 열 이름의 속성 사용  
df[ 'GDP' ] # 인덱싱에 열 이름을 사용
```

- row를 조회할 때는 df.loc[index]

DataFrame 객체 생성

- 예3) DataFrame 객체 생성하기
 - column단위로 Series를 만들고, dict에 대입해서 데이터 생성.
 - Series의 index를 DataFrame 객체의 index로 사용.

```
import pandas as pd
```

```
names = pd.Series([ 'Bob', 'Jessica','Mary','John', 'Kate' ])
births = pd.Series([ '12-25', '10-11','05-05', '06-12', '11-30' ])
members = { 'Name': names, 'Birth': births}
dfM = pd.DataFrame( members )
print(dfM)
```

출력 화면

	Name	Birth
0	Bob	12-25
1	Jessica	10-11
2	Mary	05-05
3	John	06-12
4	Kate	11-30

columns(열 이름)

index

	Name	Birth
0	Bob	12-25
1	Jessica	10-11
2	Mary	05-05
3	John	06-12
4	Kate	11-30

DataFrame 객체 생성

- 열(column) 추가하기, 행(row) 추가하기

- 예) ppt12쪽 예제에서 추가

열 추가. 열 이름 '성별'

```
dfM['성별'] = ['M','F','F','M','F']
```

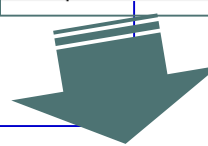
행 추가. 인덱스 이름 5

```
dfM.loc[5] = ['Hanna','11-11','F']  
print(dfM)
```

columns(열 이름)

index

	Name	Birth
0	Bob	12-25
1	Jessica	10-11
2	Mary	05-05
3	John	06-12
4	Kate	11-30



열 추가

출력 화면

	Name	Birth	성별
0	Bob	12-25	M
1	Jessica	10-11	F
2	Mary	05-05	F
3	John	06-12	M
4	Kate	11-30	F
5	Hanna	11-11	F

	Name	Birth	성별
0	Bob	12-25	M
1	Jessica	10-11	F
2	Mary	05-05	F
3	John	06-12	M
4	Kate	11-30	F
5	Hanna	11-11	F

행 추가

DataFrame 객체 생성

- pandas DataFrame은 다양한 데이터 타입을 사용해서 만들 수 있다.

- ndarray, dictionary, series, list, dataframe

2차원 배열 사용(ndarray 객체)

```
input1 = np.array([[1, 2, 3], [4, 5, 6]])  
df1= pd.DataFrame(input1)
```

columns(열 이름)

	0	1	2
0	1	2	3
1	4	5	6

index

DataFrame 객체

리스트 사용

```
data = [1,3,5,7,9]  
df3= pd.DataFrame(data, index=range(1,6),  
                  columns=['A'] )
```

columns(열 이름)

	A
1	1
2	3
3	5
4	7
5	9

index

DataFrame 객체

index

0	1
1	3
2	5
3	7
4	9

#비교

```
data = [1, 3, 5, 7, 9]  
s = pd.Series(data)
```

Series 객체 생성.

DataFrame 객체 생성

- Series 객체를 사용해서 생성하면, Series의 **index(행 이름)**를 **index**로 사용. 열 이름만 추가.

Series 객체를 사용.

```
input2 = pd.Series( {"England":"London",  
                    "India":"New Delhi", "USA":"Washington",  
                    "Belgium":"Brussels"} )
```

```
df2= pd.DataFrame(input2, columns=['city'])
```

index

England	London
India	New Delhi
USA	Washington
Belgium	Brussels

input2 (Series 객체)



Series 객체를
사용.

columns(열 이름)

	City
England	London
India	New Delhi
USA	Washington
Belgium	Brussels

생성된 DataFrame 객체

DataFrame 데이터 추출

- 데이터 엑세스 하기

- 특정 열 값을 추출: 인덱싱 또는 속성

`변수[열이름]` #지정된 열 추출

`변수[[열이름1,열이름2,...]]` #2개 이상은 리스트로 묶어 지정

`변수.속성` #속성(열 이름과 동일). 지정된 열 추출

- 리스트와 유사하게 슬라이싱으로 추출

`변수[start:end]` #start행부터 end-1행까지 추출.

- 특정 행을 추출.

`변수.loc[행이름]` #지정된 레이블의 행 추출

`변수.loc[ [행이름1,행이름2,...] ]` #2개 이상은 리스트로 묶어 지정

- 특정한 요소 하나만을 선택.

`변수.loc[ 행이름, 열이름 ]`

- boolean indexing 사용하여 특정 조건의 값들을 추출

DataFrame 데이터 추출

- 특정 열의 데이터를 가져올 때, 해당 데이터가 속한 행의 인덱스 정보도 표시

columns(열 이름)

	year	GDP_rate	GDP
index 0	2016	2.8	1.63M
1	2017	3.1	1.73M
2	2018	0.0	1.83M

시리즈

- 예) 인덱싱, 속성으로 열 추출

예제2의 객체

```
print( df['year'] ) #df.year과 동일  
print()
```

```
print( df['GDP_rate'] ) #②  
print()
```

```
print( df.GDP ) #③
```

😊 출력 화면

```
0    2016  
1    2017  
2    2018  
Name: year, dtype: int64  
  
0    2.8 #②  
1    3.1  
2    3.0  
Name: GDP_rate, dtype: float64  
  
0    1.63M #③  
1    1.73M  
2    1.83M  
Name: GDP, dtype: object
```

DataFrame 데이터 추출

- 예) boolean indexing 사용하여 데이터 엑세스
 - 'year' 열의 값이 2016보다 큰 행 추출.

```
# 예제2의 객체 .....
```

```
# boolean indexing 사용
```

```
print( df[ df['year'] > 2016 ] ) #④
```

☺ 출력 화면

	year	GDP rate	GDP	#④
1	2017	3.1	1.73M	
2	2018	3.0	1.83M	

DataFrame 연산 함수

- pandas는 데이터에 대한 연산 함수들을 제공
 - `mean()` 평균구하기
 - `median()` 중앙값 구하기
 - `sum()` 합계구하기
 - `min()` 최소를 구하기
 - `max()` 최대를 구하기
 - `corr()` 데이터프레임의 열 사이의 상관 관계를 계산
 - `count()` 각 데이터프레임 열에서 **null**이 아닌 값의 개수를 계산
 - `sort_values()` 데이터를 정렬
 - `groupby()` 기준에 따라 몇 개의 그룹으로 데이터를 분할
 - `std()` 표준편차 구하기
 - `describe()` 기본적인 통계치를 모두 표시
 - `plot()` 데이터 가시화

DataFrame 연산 함수

- 데이터를 정렬하여 액세스 하기
 - DataFrame 객체에 있는 `sort_values()`를 호출.
 - 해당 변수에 대해 정렬을 할 수 있다.(디폴트는 오름차순)
 - `ascending= False` 파라미터는 내림차순으로 정렬.

앞의 예제에서 계속.....

```
print( df.sort_values( by= 'GDP_rate' ) ) # 오름차순
```

```
print( df.sort_values( by= 'GDP_rate', ascending= False ) ) # 내림차순
```

출력 화면

	year	GDP_rate	GDP
0	2016	2.8	1.63M
2	2018	3.0	1.83M
1	2017	3.1	1.73M

오름차순

	year	GDP_rate	GDP
1	2017	3.1	1.73M
2	2018	3.0	1.83M
0	2016	2.8	1.63M

내림차순

결과에서,
인덱스가 뒤죽박죽

DataFrame 연산 함수

- 예) 액세스 결과에 인덱스 새로 설정하기
 - 기존의 인덱스가 필요없다면, **plot**을 위해 다시 설정해 줌.
`df객체.reset_index(drop=True)`
 - 결과에서 인덱스가 뒤죽박죽. 기존 데이터 셋의 인덱스를 그대로 갖고 있기 때문. **plot**을 위해 인덱스 초기화 작업을 해줌

앞의 예제에서 계속.....

```
df2 = df.sort_values(by= 'GDP rate', ascending= False)
df2 = df2.reset_index(drop=True) # 새로 인덱스 설정
print( df2 )
```

출력 화면

	year	GDP rate	GDP
0	2017	3.1	1.73M
1	2018	3.0	1.83M
2	2016	2.8	1.63M

#인덱스를 새로 설정

DataFrame 연산 함수

- 예) 특정 열의 합계 구하기

예제2 객체에서...

```
sum= df[ 'GDP_rate' ].sum()
avg= df[ 'GDP_rate' ].mean()
print(sum)
print(avg)

print()

print( df.describe() ) #②
```

변수 df

axis 1 →

axis 0 ↓

	year	GDP_rate	GDP
1	2016	2.8	1.63M
2	2017	3.1	1.73M
3	2018	3.0	1.83M

출력 화면

8.9 # sum

2.966666666666667 # mean

	year	GDP_rate#②
count	3.0	3.000000
mean	2017.0	2.966667
std	1.0	0.152753
min	2016.0	2.800000
25%	2016.5	2.900000
50%	2017.0	3.000000
75%	2017.5	3.050000
max	2018.0	3.100000

외부 데이터 파일 읽기

- pandas는 CSV 파일, 엑셀 파일, 텍스트 파일 등 다양한 외부 데이터를 읽고 쓸 수 있는 기능을 제공한다.
- 엑셀 파일(확장자 xlsx)을 읽어 DataFrame 객체를 만들기
 - 엑셀파일을 dataframe 객체로 만들려면 openpyxl 엔진을 활용한다.
pip install openpyxl
 - pd.read_excel('파일이름')
- CSV 파일 읽고 객체 만들기.
 - pd.read_csv('파일이름')
- 그래프 그리기
 - 데이터를 pandas를 통해 읽어 들인 후, matplotlib 라이브러리 패키지를 통해 그래프를 그릴 수 있다.
 - matplotlib.pyplot 모듈을 import 한다

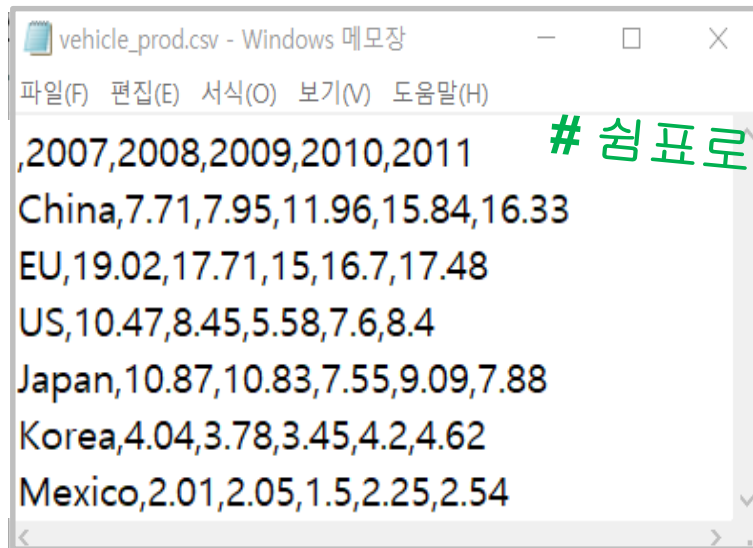
```
import pandas as pd  
df= pd.read_excel( 'F:\수강생.xlsx' )
```

읽은 파일을 df에 할당

```
from matplotlib import pyplot as plt  
# 또는  
import matplotlib.pyplot as plt
```


CSV 파일 읽기

- CSV 파일 읽고 DataFrame 객체를 만들기
 - CSV파일은 구분자(쉼표)로 데이터를 구분하여, 필드를 나타내는 열과 레코드를 나타내는 행으로 구성된다.
 - `pd.read_csv('파일이름')`



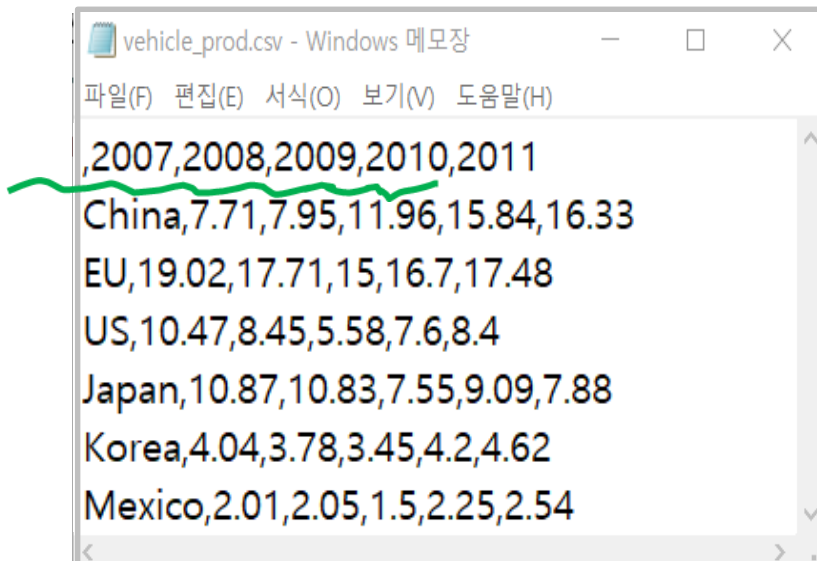
데이터 출처:

<https://github.com/dongupak/DataML/tree/main/csv>

CSV 파일 읽기

- CSV 파일 읽고 DataFrame 객체를 만들기
 - CSV파일은 구분자(쉼표)로 데이터를 구분하여, 필드를 나타내는 열과 레코드를 나타내는 행으로 구성된다.
 - `pd.read_csv('파일이름')`
- 예) csv 파일(주요 자동차 생산 국가별 자동차 생산 대수)
 - 첫번째 열을 인덱스로 사용하기 위해, 첫번째 열의 이름을 생략.

vehicle_prod.csv



첫번째 열의 이름을 생략.

	A	B	C	D	E	F
1		2007	2008	2009	2010	2011
2	China	7.71	7.95	11.96	15.84	16.33
3	EU	19.02	17.71	15	16.7	17.48
4	US	10.47	8.45	5.58	7.6	8.4
5	Japan	10.87	10.83	7.55	9.09	7.88
6	Korea	4.04	3.78	3.45	4.2	4.62
7	Mexico	2.01	2.05	1.5	2.25	2.54

메모장에서 생성(CSV파일)

CSV 파일 읽기

- 예) csv파일 읽기

```
import pandas as pd
v= pd.read_csv( 'F:/vehicle_prod.csv' )
print(v)#①
```

😊출력 화면

#①

Unnamed: 0		2007	2008	2009	2010	2011
0	China	7.71	7.95	11.96	15.84	16.33
1	EU	19.02	17.71	15.00	16.70	17.48
2	US	10.47	8.45	5.58	7.60	8.40
3	Japan	10.87	10.83	7.55	9.09	7.88
4	Korea	4.04	3.78	3.45	4.20	4.62
5	Mexico	2.01	2.05	1.50	2.25	2.54

#각 행의 번호가 자동으로 부여. 0~5

#비워 두었던 첫 열은 Unnamed:0이라는 이름이 부여됨

CSV 파일 읽기

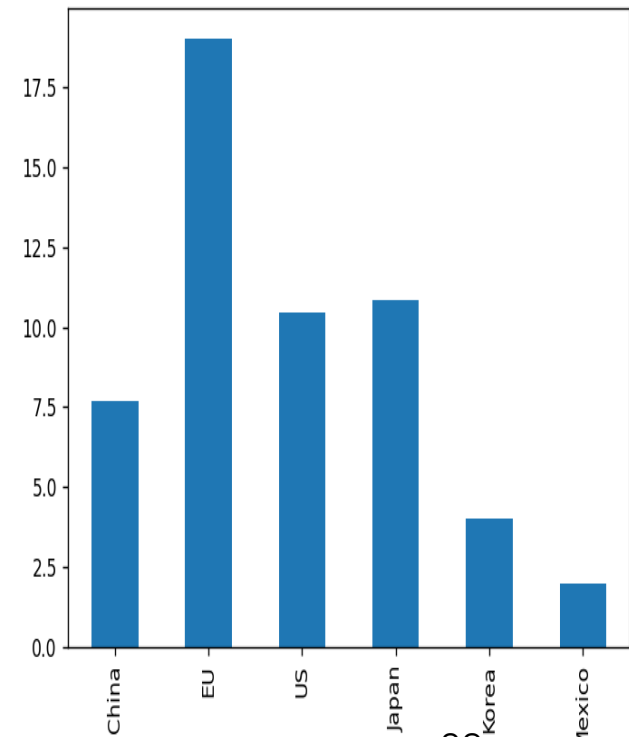
- 예) csv파일 읽기
 - 국가 이름이 있는 0번 열을 인덱스로 사용하도록 설정.

```
import pandas as pd
v2= pd.read_csv( 'F:/vehicle_prod.csv', index_col=0 )
print(v2)
v2["2007"].plot(kind='bar') #열'2007' 시각화
plt.show()
```

😊출력 화면

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

#첫번째 열이 인덱스로 사용



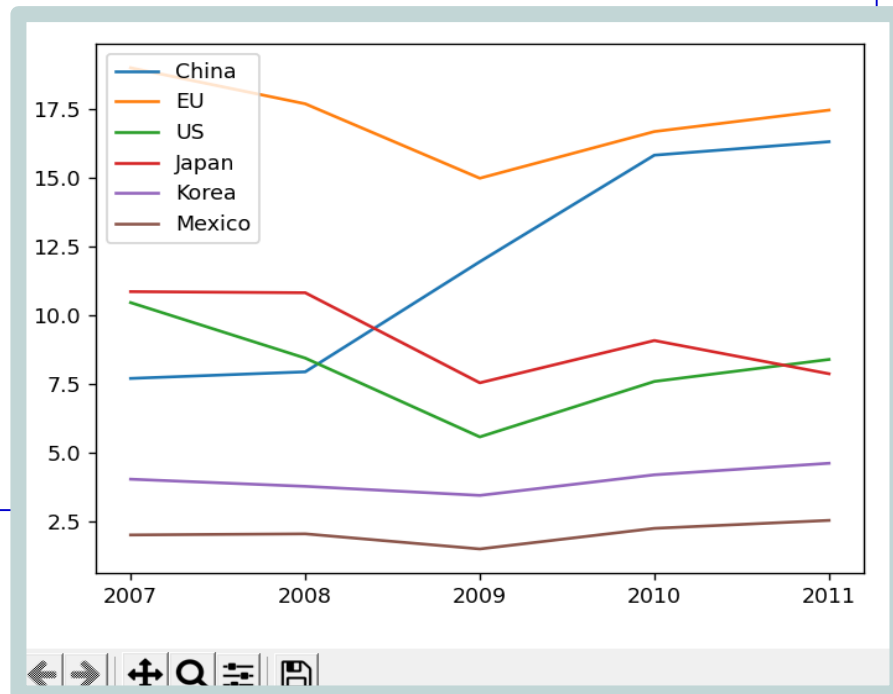
CSV 파일 읽기

- 예) csv파일 읽고, 그래프 그리기
 - 국가별로 년도 별 생산량 그래프 그리기
 - x축- 열 이름을 표시,
 - y축- 행(년도 별 생산량)의 값을 표시

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

```
import pandas as pd
v2= pd.read_csv( 'F:/vehicle_prod.csv', index_col=0 )
plt.legend(["China", "EU", "US", "Japan", "Korea", "Mexico"])
```

```
plt.plot(v2.loc["China"])
plt.plot(v2.loc["EU"])
plt.plot(v2.loc["US"])
plt.plot(v2.loc["Japan"])
plt.plot(v2.loc["Korea"])
plt.plot(v2.loc["Mexico"])
plt.show()
```



CSV 파일 읽기

- 예) csv파일 읽은 후 통계 구하기
 - 국가별 합계와 평균을 구해보자.
 - `sum(axis=1)` **#axis=1 행방향 합계**
 - `mean(axis=1)`

	2007	2008	2009	2010	2011	
China	7.71	7.95	11.96	15.84	16.33	11.958
EU	19.02	17.71	15.00	16.70	17.48	17.182
US	10.47	8.45	5.58	7.60	8.40	⋮
Japan	10.87	10.83	7.55	9.09	7.88	
Korea	4.04	3.78	3.45	4.20	4.62	
Mexico	2.01	2.05	1.50	2.25	2.54	2.070

☺실행 화면

```
>>> v2.sum(axis=1)
```

```
China      59.79
EU         85.91
US         40.50
Japan      46.22
Korea      20.09
Mexico     10.35
dtype: float64
```

☺실행 화면

```
>>> v2.mean(axis=1)
```

```
China      11.958
EU         17.182
US          8.100
Japan       9.244
Korea       4.018
Mexico      2.070
dtype: float64
```

#국가별(행 방향) 합계, 평균 계산

CSV 파일 읽기

- 예) csv파일 읽은 후 열 추가하기
 - 국가별 합계와 평균을 구해서 열을 추가.
 - `sum(axis=1)` # **axis=1** 행방향 합계

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

```
>>> # 합계를 구하고, 열 추가
>>> v2['total']= v2.sum( axis=1 )
>>> #국가별 평균은 열'2007'~ 열'2011'의 값으로 계산
>>> v2['평균']=v2[ ['2007','2008','2009','2010','2011'] ].mean(axis=1)
>>> print(v2)
```

😊출력 화면

axis 1

#열 추가

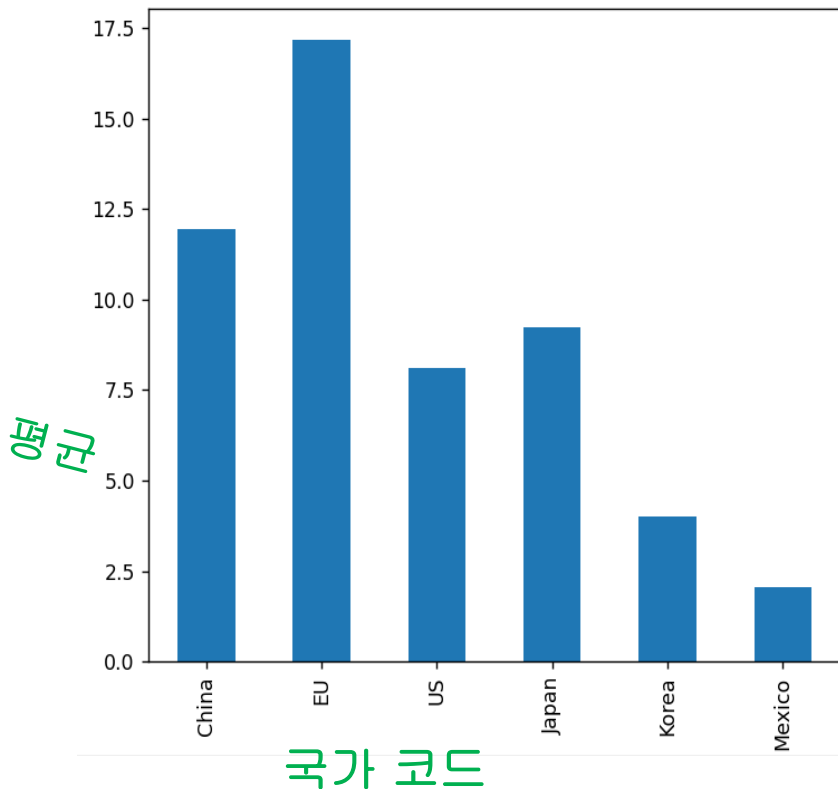
	2007	2008	2009	2010	2011	total	평균
China	7.71	7.95	11.96	15.84	16.33	59.79	11.958
EU	19.02	17.71	15.00	16.70	17.48	85.91	17.182
US	10.47	8.45	5.58	7.60	8.40	40.50	8.100
Japan	10.87	10.83	7.55	9.09	7.88	46.22	9.244
Korea	4.04	3.78	3.45	4.20	4.62	20.09	4.018
Mexico	2.01	2.05	1.50	2.25	2.54	10.35	2.070

axis 0

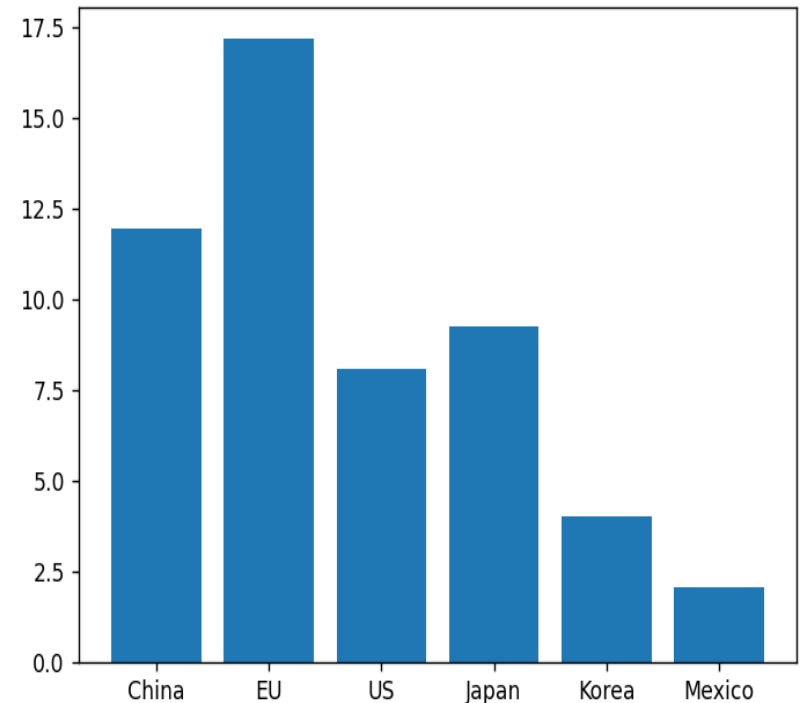
CSV 파일 읽기

- 추가된 열 ‘평균’을 시각화하기. 국가별 평균을 그래프로 표현
 - x축-인덱스(국가 코드)가 표시, y축-평균값 표시

```
v2['평균'].plot(kind='bar')  
plt.show()
```



```
plt.bar(v2.index, v2['평균'])  
plt.show()
```



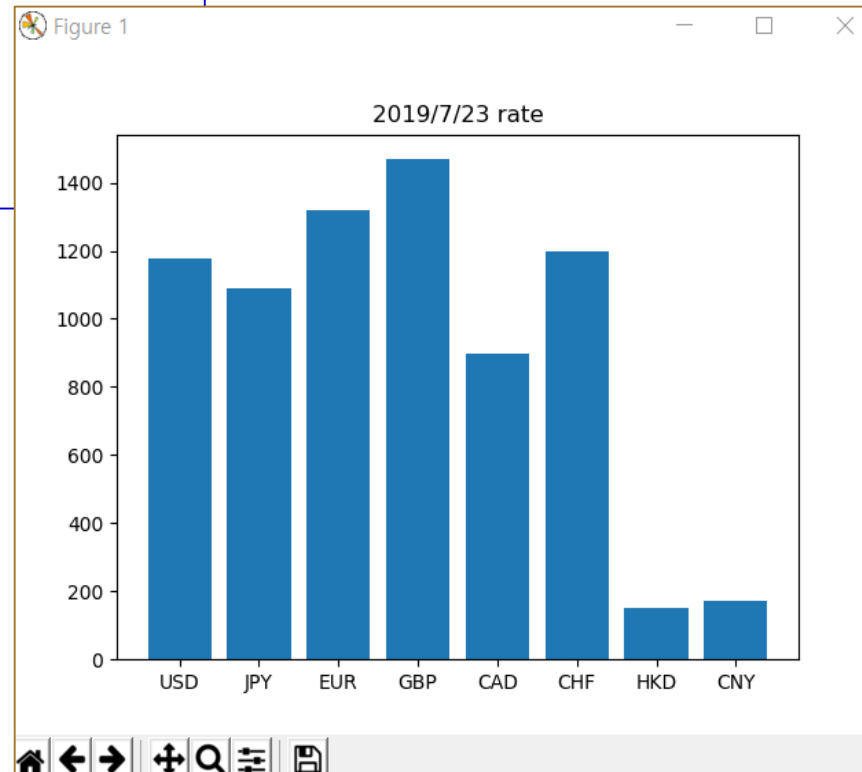
엑셀 파일 읽기

- 예) 엑셀파일을 읽어 통화별 매매기준율의 bar 차트 그리기

```
import pandas as pd
import matplotlib.pyplot as plt
df= pd.read_excel( 'F:\일일환율표.xlsx' )
```

```
plt.bar( df.통화표시, df.매매기준율 )
plt.title( '2019/7/23 rate' )
plt.show()
```

출력 화면 (그래프)



샘플 엑셀 파일 (일일환율표.xlsx)

	A	B	C	D
1	통화표시	통화명	매매기준율	한국은행기준율
2	USD	미국 달러	1178.5	1177.6
3	JPY	일본 100엔	1089.39	1091.33
4	EUR	유럽연합 유로	1319.37	1319.91
5	GBP	영국 파운드	1468.28	1469.06
6	CAD	캐나다 달러	897.3	897.25
7	CHF	스위스 프랑	1197.05	1198.7
8	HKD	홍콩 달러	150.88	150.79
9	CNY	중국 위안	171.17	171.08

엑셀 파일 읽기

- 예) 날짜별 달러&유로화의 매매기준율을 그래프로 그리기

```
import pandas as pd
import matplotlib.pyplot as plt

df= pd.read_excel( 'F:\환율.xlsx' )
print(df) #①

plt.plot( df['USD'] ) #②
# plt.plot( df.USD ) 와 동일
plt.show()
```

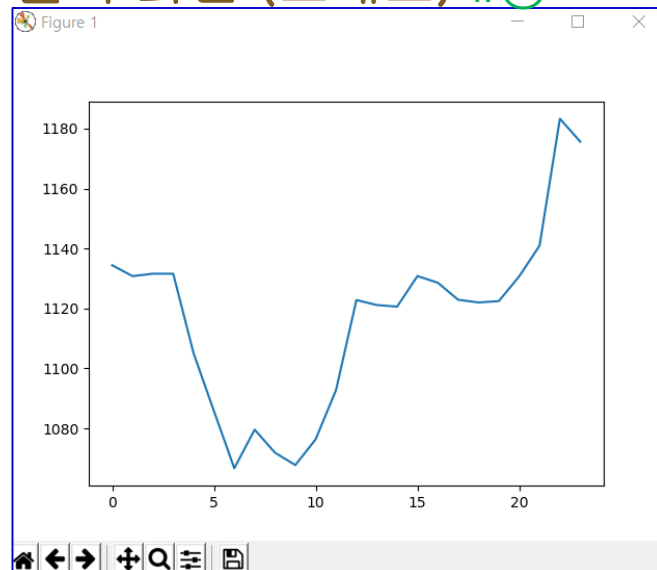
출력 화면(shell) #①

	날짜	USD	EUR(유로화)
0	2017.07	1134.40	1305.93
1	2017.08	1130.79	1336.51
2	2017.09	1131.59	1347.86
3	2017.10	1131.57	1331.28
4	2017.11	1105.04	1296.16
5	2017.12	1085.78	1284.44
6	2018.01	1066.70	1299.57
7	2018.02	1079.58	1332.23
8	2018.03	1071.89	1322.89
9	2018.04	1067.76	1311.84
10	2018.05	1076.39	1271.80
11	2018.06	1092.80	1275.06
12	2018.07	1122.80	1312.03
13	2018.08	1121.15	1296.08
14	2018.09	1120.60	1304.74
15	2018.10	1130.81	1299.55
16	2018.11	1128.58	1282.44
17	2018.12	1122.90	1277.00
18	2019.01	1122.00	1281.52
19	2019.02	1122.45	1272.47
20	2019.03	1130.72	1278.47
21	2019.04	1140.95	1281.77
22	2019.05	1183.29	1323.65
23	2019.06	1175.62	1327.32

샘플 엑셀 파일 (환율.xlsx)

	A	B	C
1	날짜	USD	EUR(유로화)
2	2017-07	1134.40	1305.93
3	2017-08	1130.79	1336.51
4	2017-09	1131.59	1347.86
5	2017-10	1131.57	1331.28
6	2017-11	1105.04	1296.16

출력 화면 (그래프) #②



엑셀 파일 읽기

- 예) 플롯에 2개의 라인을 추가하기

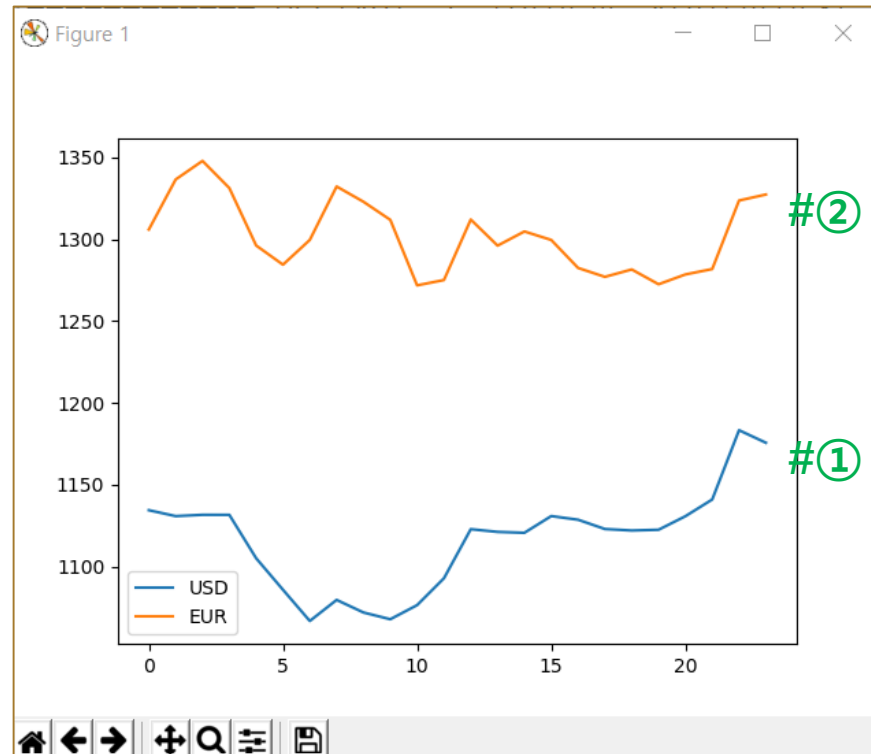
```
plt.plot( df.USD ) #①  
plt.plot( df['EUR(유로화)] ) #②  
plt.legend( ['USD', 'EUR'] ) # 범례 추가  
plt.show()
```

출력 화면 (그래프)

샘플 엑셀 파일 (환율.xlsx)

	A	B	C	D
1	날짜	USD	EUR(유로화)	
2	2017.07	1134.40	1305.93	
3	2017.08	1130.79	1336.51	
4	2017.09	1131.59	1347.86	
5	2017.10	1131.57	1331.28	
6	2017.11	1105.04	1296.16	
7	2017.12	1085.78	1284.44	
8	2018.01	1066.70	1299.57	
9	2018.02	1079.58	1332.23	
10	2018.03	1071.89	1322.89	
11	2018.04	1067.76	1311.84	
12	2018.05	1076.39	1271.80	

파일의 일부 데이터



엑셀 파일 읽기

- 예) 엑셀 파일의 데이터를 읽어 그래프 그리기

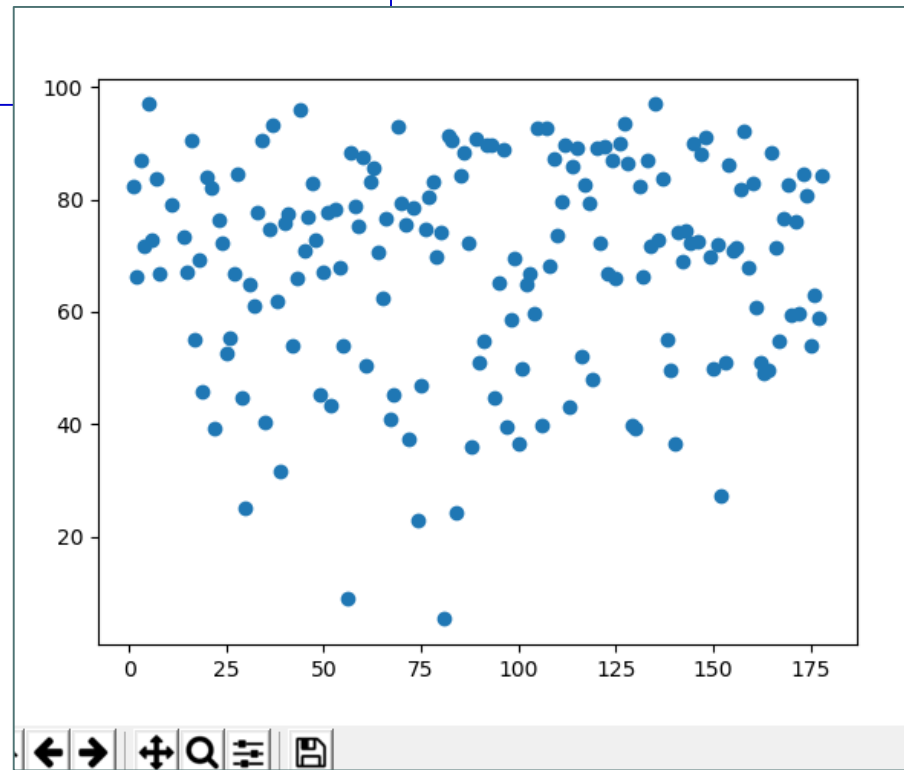
```
import matplotlib.pyplot as plt  
df= pd.read_excel( 'F:\성적.xlsx' )
```

```
plt.scatter( df.번호, df.점수 )  
plt.title( '2019/7/23 rate' )  
plt.show()
```

출력 화면 (그래프)

샘플 파일 (성적.xlsx)

	A	B
1	번호	점수
2	1	82.2
3	2	66.3
4	3	87.0
5	4	71.7
6	5	96.9
7	6	72.8
8	7	83.7
9	8	66.8
10	11	79.1
11	14	73.4
12	15	67.2
13	16	90.4



파일의 일부 데이터

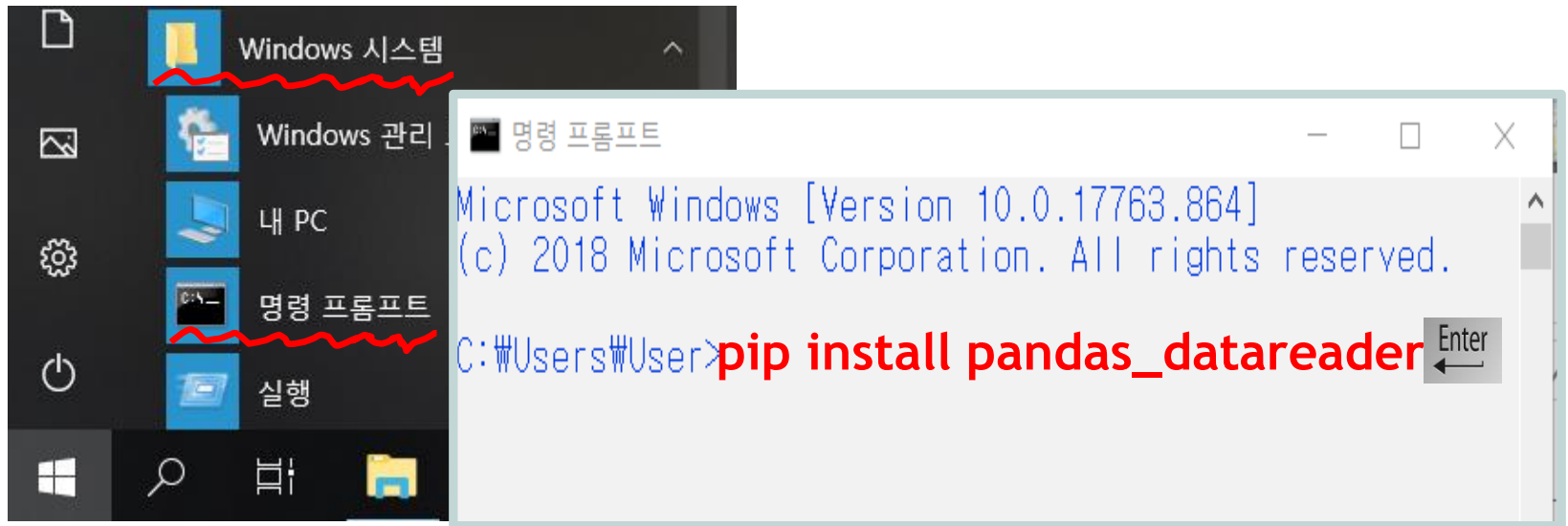
DataReader로 Web 데이터 가져오기

- 웹 상의 데이터를 DataFrame 객체로 만들기.
 - `pandas_datareader` 패키지의 `data` 모듈 안에 있는 `DataReader()` 함수 사용.
 - 이 함수는 웹 상의 데이터를 DataFrame 객체로 만들어 반환.
 - `pandas_datareader` 패키지 설치 및 사용하기
 - `pip install pandas_datareader` #설치하기
 - `import pandas_datareader.data as web` #import
 - `data` 모듈을 import 하고, `DataReader()` 함수를 사용
- ```
web.DataReader(종목, 데이터를 가져올 소스, 조회 시작일, 종료일)
```
- `DataReader()`에 조회 시작일, 종료일을 인수 값으로 지정.
    - `datetime` 모듈은 파이썬에서 날짜를 표현하는 데 사용.
      - `datetime()`을 사용하여, 날짜를 `datetime` 객체로 만들어 지정.
      - 주의: 1~9월은 입력할 때 01, 09 아니라 1, 9로 입력해야 함.
      - (예) `datetime.datetime(2021,11,1)` #2021년 11월 1일
    - 'yyyy-mm-dd'로 구성해서 지정해도 됨. (예) '2021-11-01'

# DataReader로 Web 데이터 가져오기

- pandas\_datareader를 윈도우 커맨드창에서 install하기
  - 윈도우 콘솔창은 <Windows 시스템> → <명령 프롬프트>

pip install pandas\_datareader # install 명령어



# DataReader로 Web 데이터 가져오기

- 예) 야후 파이낸스로부터 GS 종목의 데이터를 일부 가져오기.
  - 조회 시작 날짜(2021-11-01)와 조회 종료 날짜(2021-11-15)를 datetime 객체로 만들어 사용한다.  
year, month, day, hour, minute, second 속성이 있음
  - GS의 종목코드는 '078930'
  - DataReader() 함수에 인수들을 지정한다.

```
import pandas_datareader.data as web
import datetime
import matplotlib.pyplot as plt

start = datetime.datetime(2018, 2, 19) #인수(year,month,day)
end = datetime.datetime(2018, 3, 4)
#또는 start='2018-02-19'; end='2018-03-04'

gs = web.DataReader("078930.KS", "yahoo", start, end)
print(gs.info())
```

<https://finance.yahoo.com/>

# DataReader로 Web 데이터 가져오기

- 실행화면
  - DataReader() 함수가 DataFrame 객체를 반환하고, 이 값이 변수 gs에 바인딩되어 있음.  
DataFrame 객체가 생성된 것을 볼 수 있다.

## 출력 화면

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 10 entries, 2018-02-19 to 2018-03-05
Data columns (total 6 columns):
High 10 non-null int64
Low 10 non-null int64
Open 10 non-null int64
Close 10 non-null int64
Volume 10 non-null int64
Adj Close 10 non-null float64
dtypes: float64(1), int64(5)
memory usage: 560.0 bytes
None
```



# DataReader로 Web 데이터 가져오기

- DataReader 함수가 반환한 DataFrame 객체를 할당받은 변수 **gs**의 값을 확인해보자.

```
>>> gs
```

| Date       | High  | Low   | Open  | Close | Volume | Adj Close    |
|------------|-------|-------|-------|-------|--------|--------------|
| 2018-02-19 | 64700 | 63700 | 64300 | 64000 | 125302 | 61620.355469 |
| 2018-02-20 | 64300 | 63200 | 63600 | 63600 | 78574  | 61235.226562 |
| 2018-02-21 | 64100 | 62400 | 64100 | 63600 | 200479 | 61235.226562 |
| 2018-02-22 | 63500 | 62200 | 63000 | 62700 | 162513 | 60368.691406 |
| 2018-02-23 | 64700 | 62700 | 63000 | 64600 | 156087 | 62198.042969 |
| 2018-02-26 | 67700 | 64700 | 64700 | 67100 | 266703 | 64605.089844 |
| 2018-02-27 | 67800 | 66000 | 66900 | 66200 | 147860 | 63738.554688 |
| 2018-02-28 | 66800 | 65300 | 65400 | 66100 | 167589 | 63642.273438 |
| 2018-03-02 | 65900 | 64100 | 65000 | 65200 | 128556 | 62775.734375 |
| 2018-03-05 | 67200 | 64800 | 64800 | 66300 | 189233 | 63834.835938 |

```
>>>
```

# DataReader로 Web 데이터 가져오기

- (예) 열 이름 'Close'의 데이터를 가지고 그래프를 그려보자

```
plt.plot(gs['Close'])
plt.show()
```

출력 화면 (그래프)

